Academic Research Navigator

UCS503 Software Engineering Project Report
Mid-Semester Evaluation

Submitted by:
(1024030385) Suukrat Agrawal
(1024030376) Krishu Shahu
(1024030388) Inderpal Singh
(102303432) Aditya Verma

BE Third Year, COE

Submitted to:
Dr. Stuti Chugh

THAPAR INSTITUTE
OF ENGINEERING & TECHNOLOGY
(Deemed to be University)

Computer Science and Engineering Department
TIET, Patiala

Table of Contents

1 Project Selection Phase

1.1 Software Bid — why this project

The Academic Research Navigator (ARN) addresses a common difficulty in academic research: finding useful material often starts with a keyword search, while a student's actual question may contain several related ideas. The project therefore focuses on building a small discovery layer that can take a plain-language research query, identify recognizable concepts, rank catalogue resources, and provide a short reason for each returned result.

The problem is suitable for a Software Engineering project because the first useful increment can be implemented and demonstrated without requiring a production-scale research infrastructure. The current prototype separates data loading, concept matching, retrieval, explanations, and relationship lookup so that these parts can be changed independently as the project moves from a development dataset to the intended university catalogue.

Problem being addressed

- Conventional keyword search can require the student to guess the wording used in catalogue metadata.
- A research question can contain concepts that are related even when the student does not explicitly know the catalogue's preferred subject terms.
- A ranked result is more useful when the student can see an understandable reason for its relevance.
- Related-topic exploration can help a student continue discovery without repeatedly con-structing a new query.
- The prototype needs to work before the final TIET library/repository data export is available.

Proposed solution

Build a browser-based search and discovery prototype with a Python/FastAPI backend. The current implementation accepts a free-text query, identifies concepts using a controlled vocabulary and aliases, ranks resources using TF-IDF cosine similarity with a small concept-overlap boost, generates template-based relevance explanations, and supports related-concept exploration through co-occurrence counts.

Expected value at this stage

- A working end-to-end research search flow rather than a static UI mock-up.
- Ranked results instead of a simple yes/no keyword match.
- Traceable concept matches and short explanations for returned resources.
- A lightweight related-concept mechanism based on the catalogue data.
- A replaceable data-loading boundary for the eventual TIET export.

Feasibility

- **Technical:** The prototype uses FastAPI, scikit-learn TF-IDF/cosine similarity, and a browser frontend implemented in a single HTML/CSS/JavaScript file.
- **Data:** The current backend reads data/catalogue.json; the file contains 6,398 book records and is labelled *Kaggle Books Dataset*.
- **Operational:** The current flow is intentionally small: a student enters a query, reviews ranked results, filters/sorts them, and explores related concepts or a resource detail view.
- **Scope:** Institutional authentication, live TIET data and multi-source retrieval are not part of the current implementation.

1.2 Project Overview — Academic Research Navigator

ARN is a concept-aware academic resource discovery prototype. A student enters a research topic or question in plain language. The system identifies matching terms from a controlled vocabulary, retrieves catalogue resources, ranks them, and returns related concepts that can be explored further. The current implementation demonstrates this flow over one local catalogue dataset rather than over live university systems.

The current browser interface includes research-area tabs for AI and Machine Learning, Computer Science, Electronics and Communication, Data and Analytics, Sciences, and Management and Business. The cards under these tabs place an example query into the search box and start a search. Search results can then be filtered by resource type and year and sorted by relevance, newest, or oldest. Selecting a result opens a detail modal containing the metadata available from the backend and, where available, links.

The backend loads the catalogue once at startup. Each resource is converted into searchable text from its title, abstract, subjects, and keywords, and is also assigned concept tags using the same deterministic phrase/alias matching approach used for queries. The retrieval module fits a TF-IDF vectorizer over this catalogue text and computes cosine similarity for each query. A small additional score of 0.15 per overlapping identified concept is then applied before the top ten results are returned.

Data status. The supplied implementation does not connect to a live TIET library or institutional repository. data/catalogue.json is the active data source and contains 6,398 book records. The real TIET export remains future scope. The loader is isolated so that a later export can be mapped into the same fields: id, title, author, year, type, subjects, keywords, and abstract.

2 Analysis Phase

2.1 Use Cases

The prototype has one primary actor: the Student. Administrative or librarian-facing workflows are outside the current implementation. The use cases therefore describe discovery behaviour rather than account management or institutional library operations.

2.1.1 Use-Case Diagram

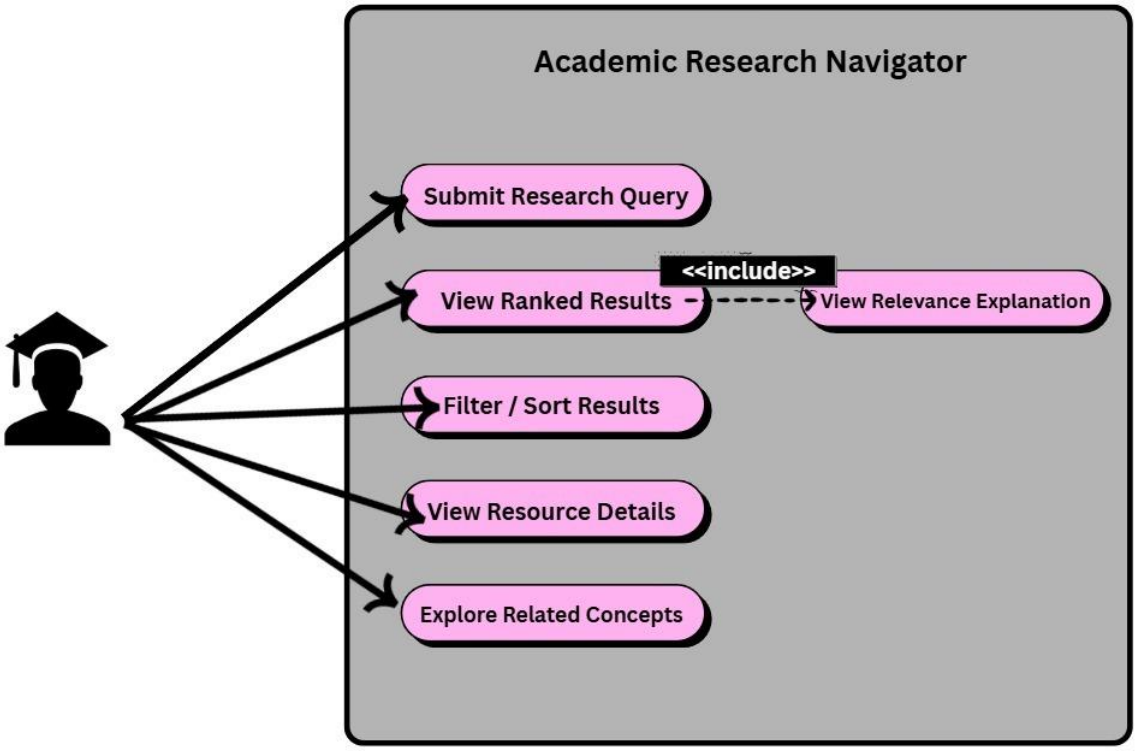

Figure 1: Use-case view of the Academic Research Navigator prototype.

The diagram groups the current user-facing functions into five actions: submitting a research query, viewing ranked results, filtering/sorting results, viewing resource details, and exploring related concepts. Viewing the relevance explanation is shown as an included part of viewing ranked results because the backend generates an explanation for each search result.

2.1.2 Use Case Templates

Use Case	**Submit Research Query**
Actor	Student
Precondition	Student has a research topic or question.
Main flow	1. Student enters a topic/question. 2. Frontend sends a POST request to /api/search. 3. Backend identifies concepts. 4. Backend retrieves and ranks resources. 5. Backend returns ranked results, explanations and related concepts. 6. Frontend displays the results and concept panel.
Postcondition	Ranked results and related concepts are displayed.
Alternate flow	If the query is empty, the frontend asks the student to enter a topic. If no resources receive a positive score, the result area shows an empty state. If no concepts are identified, retrieval still uses TF-IDF similarity.

Use Case	**Explore Related Concept**
Actor	Student
Precondition	Student is viewing search results or a concept associated with the current discovery flow.
Main flow	1. Student selects a concept chip. 2. Frontend requests /api/concept/{concept}. 3. Backend returns resources tagged with that concept and related concepts based on co-occurrence counts. 4. Frontend renders the new result set.
Postcondition	Student sees resources and related concepts without typing a new query.
Alternate flow	If the concept is not present in the relationship index, the API returns a not-found response.

Use Case	**Filter / Sort Results**
Actor	Student
Precondition	A result list has been returned.
Main flow	1. Student chooses a resource type and/or year. 2. Student chooses relevance, newest, or oldest. 3. Frontend applies the selected options to the returned result list and redraws the cards.
Postcondition	Only results satisfying the selected filters are shown, in the selected order.

Use Case	**View Resource Detail**
Actor	Student
Precondition	A resource is present in the current result list.
Main flow	1. Student selects a result. 2. Frontend opens the resource detail view. 3. The view presents title, author, year, type, identifier and available metadata, subjects, abstract, and relevance explanation.
Postcondition	Student can inspect the resource metadata without leaving the discovery interface.
Alternate flow	Missing fields are shown as unavailable/not provided; the current catalogue records do not contain working TIET biblionumber links.

2.2 Activity Diagram and Swimlane Diagram

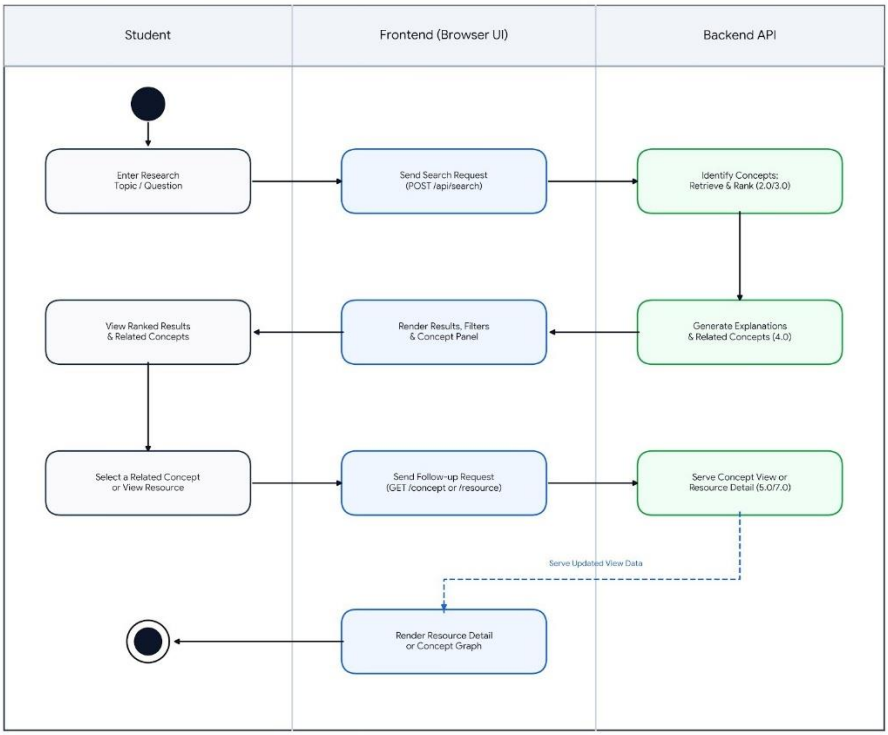

Figure 2: Activity/swimlane view of the current search and exploration flow.

The diagram follows three lanes: Student, Frontend (browser UI), and Backend API. The main path starts with a research query, moves through the POST search request, concept identification and retrieval/ranking, and returns results to the browser. A follow-up request is then possible when the student selects a related concept or opens a resource detail view.

2.3 Data Flow Diagrams (DFDs)

2.3.1 DFD Level 0

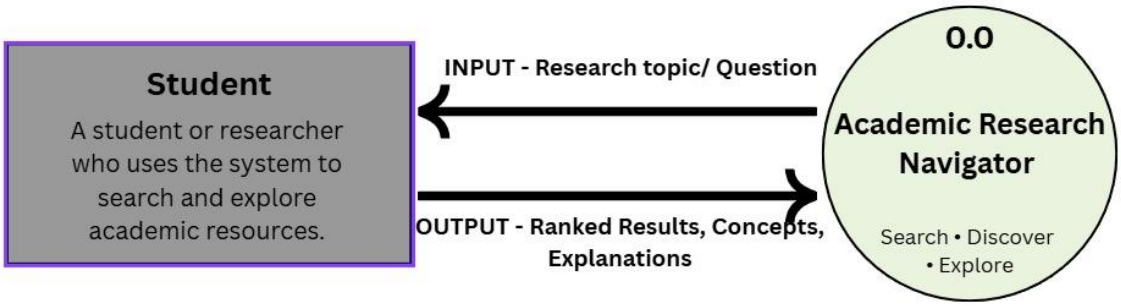

Figure 3: DFD Level 0: the student supplies a research topic/question and receives

ranked resources, concepts and explanations.
At the context level, the Student is the external entity and the Academic Research Navigator is the single system process. The current prototype does not show separate institutional repositories or external scholarly databases because no such runtime integrations are implemented.

2.3.2 DFD Level 1

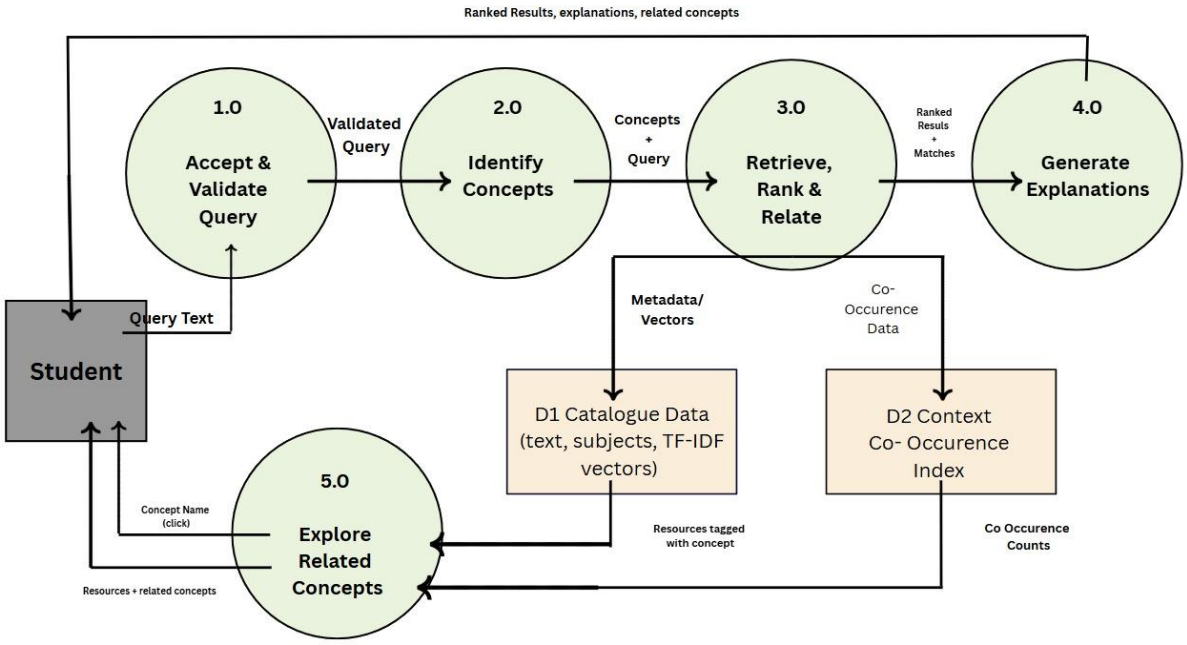

Figure 4: DFD Level 1: decomposition of the implemented discovery pipeline.

The Level 1 view corresponds to the backend modules. Process 2.0 is concept identification, Process 3.0 is retrieval/ranking plus the relationship lookup used for the response, Process 4.0 is explanation generation, and Process 5.0 is related-concept exploration. D1 represents the catalogue data and the derived TF-IDF representation used during runtime; D2 represents the in-memory concept co-occurrence index. These structures are built when the application starts from the local catalogue file.

2.3.3 DFD Level 2

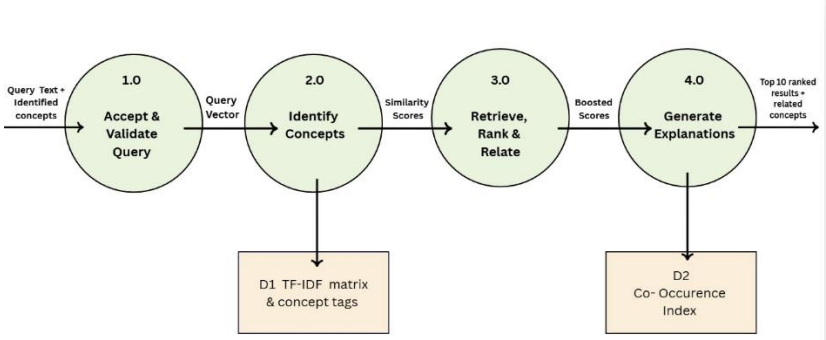

Figure 5: DFD Level 2: internal steps of retrieval, ranking and related-concept selection.

The Level 2 diagram decomposes retrieval into query vectorization, cosine-similarity scoring, concept-overlap boosting, and final ranking/top-k selection. The implementation uses the same TF-IDF vectorizer fitted on the catalogue for both catalogue documents and incoming queries. Concept overlap adds 0.15 for each shared identified concept, after which positive-scoring resources are sorted and the top ten are returned.

2.4 Software Requirement Specification

This SRS is intentionally abridged for the prototype stage. Requirements below distinguish the behaviour present in the supplied implementation from features that remain future scope.

2.4.1 Introduction

Purpose. The SRS defines the functional behaviour and quality expectations of the current ARN prototype.

Scope. The implemented scope covers query intake, deterministic concept identification, catalogue retrieval and ranking, relevance explanation, related-concept exploration, result filtering/sorting in the browser, and resource-detail display.

Definitions. A *concept* is a term from the controlled vocabulary or an alias mapped to one of its canonical terms. A *resource* is one record in the local catalogue containing fields such as title, author, year, type, subjects, keywords and abstract.

2.4.2 Overall Description

The system is a client-server web application. The frontend is a browser-based HTML/CSS/-JavaScript interface and communicates with a Python/FastAPI backend through REST-style HTTP endpoints. At startup, the backend loads data/catalogue.json, derives concept tags, fits the TF-IDF representation, and builds the concept co-occurrence index in memory.

2.4.3 Functional Requirements

ID	Requirement
FR-1	The system shall accept a non-empty free-text research query.
FR-2	The system shall identify concepts using controlled-vocabulary phrase matching and aliases.
FR-3	The system shall rank catalogue resources using TF-IDF cosine similarity.
FR-4	The system shall add a concept-overlap boost to resources sharing identified query concepts.

FR-5	The system shall return resources associated with a selected concept.
FR-6	The system shall return related concepts ranked by co-occurrence count.
FR-7	The frontend shall allow filtering by resource type and year and sorting by relevance, newest, or oldest.
FR-8	The system shall generate a short explanation for each returned search result using observable metadata matches.
FR-9	The system shall provide a resource-detail view containing available meta-data, subjects and abstract/description.
FR-10	The system shall expose a health endpoint reporting service status, resource count and data-source label.

2.4.4 Non-Functional Requirements

ID	Requirement / current interpretation
NFR-1	**Explainability:** concept matches and relevance explanations should be traceable to matched phrases, shared concepts or subject terms.
NFR-2	**Swappability:** catalogue loading is isolated in data_loader.py, provid-ing a clear boundary for replacing the current dataset.
NFR-3	**Maintainability:** concept matching, retrieval, relationship counting and explanation generation are separated into modules.
NFR-4	**Extensibility:** a future retrieval method can be compared with the TF-IDF baseline without changing the browser interaction model.
NFR-5	**Prototype resilience:** the current discovery pipeline operates entirely on local catalogue data and does not depend on a live external scholarly service.

2.4.5 External Interface Requirements

Interface	Current interaction
Browser UI	Search box, research-area cards/tabs, result cards, filters, sort controls, related-concept chips and resource detail modal.
Search API	POST /api/search with a JSON body containing query.
Concept API	GET /api/concept/{concept}.
Resource API	GET /api/resource/{id} and an HTML resource view at GET /api/resource/\{id\}/view.
Health API	GET /api/health.
Concept utility	GET /api/concepts, used as a utility/debugging endpoint rather than a main user workflow.

Data source	Local JSON file data/catalogue.json; no live TIET cata- logue/repository connection is present.

3 User Stories and Story Cards

The stories below are limited to interactions supported by the current prototype.

ID	Story
US-1	As a student, I want to search using a plain-language description of my topic so that I can start with the question I actually have.
US-2	As a student, I want to see why a result was considered relevant so that I can judge the returned resources more quickly.
US-3	As a student, I want to explore related concepts so that I can continue discovery without writing a completely new query.
US-4	As a student, I want to filter and sort results by type and year so that I can narrow the returned list.
US-5	As a student, I want to open a resource detail view so that I can inspect its metadata and abstract/description.
US-6	As a developer, I want the catalogue loader to be isolated so that the real university export can be integrated without rewriting retrieval and relationship logic.

Story Card — US-2: View Relevance Explanation

Priority	High
Need	Show a short reason for why a returned resource is relevant.
Acceptance criteria	Each search result contains an explanation generated from its matched concepts; when no concept overlap exists, the imple-mentation falls back to subject terms or a generic overlapping-term statement.
Dependency	Search result metadata and the concept/retrieval output.

4 Supporting Design Views

a. System Architecture

The current architecture is intentionally small. The browser frontend communicates with the FastAPI application. The application coordinates four focused modules and the data loader. The catalogue file is read at startup; the resulting resources are used to build the TF-IDF matrix and relationship index in memory.

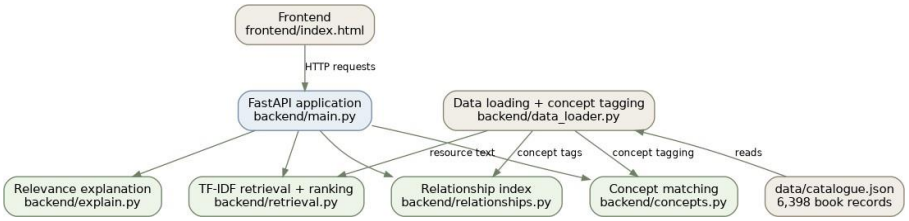

Figure 6: Module-level architecture derived from the supplied implementation.

b. Module Structure

Module	Responsibility
main.py	Creates the FastAPI application, loads resources/indexes, defines the API routes, validates the search request, and serializes response data.
data_loader.py	Reads data/catalogue.json, constructs searchable text, assigns concept tags and returns resources plus the source label.
concepts.py	Normalizes text, performs controlled-vocabulary phrase matching and alias resolution, and returns canonical concepts.
retrieval.py	Fits TF-IDF over resource text, computes cosine similarity, applies concept-overlap boosting and returns the top positive-scoring results.
relationships.py	Builds concept-to-resource lists and concept co-occurrence counts and returns related resources/concepts.
explain.py	Produces short template-based relevance explanations from matched concepts or subject terms.
frontend/index.html	Provides the browser UI, sends API requests, renders results/concepts, applies local filters/sorting and opens resource details.

c. Data Model and Data Preparation

The active catalogue contains 6,398 records. All records are of type *Book*. The observed metadata fields are id, title, author, year, type, subjects, keywords, and abstract. The source label stored in the JSON file is *Kaggle Books Dataset*.

At startup, the loader joins title, abstract, subjects and keywords into a searchable text field. The concept module then applies deterministic matching to this text. The controlled vocabulary in the supplied implementation contains 74 canonical terms, with aliases for common surface variants such as “sci-fi”, “YA”, “engineering”, “sports”, and “business”. This is a correction to the earlier report wording that described the

list as 34 hand-typed terms: the current code contains 74 terms and its comment states that the vocabulary was derived from subject frequencies in the 6,398-record catalogue.
The dataset should be treated as a development stand-in. It demonstrates the pipeline but does not provide evidence about coverage of TIET's actual library or repository resources. The current data also does not provide biblionumber values, so working TIET catalogue-record links cannot be claimed.

5 SDLC Model Adopted

The project follows an Agile, iterative-incremental approach rather than a fixed waterfall sequence. This is appropriate because the data layer is still subject to change and the team can validate the search workflow in small increments.
The first increment is the current prototype: a working search-and-explore flow over the local catalogue. The approach allows the team to establish a baseline with TF-IDF and deterministic concept matching before deciding whether a more complex retrieval approach is actually useful. Future iterations can replace the data loader when the real export arrives and can compare alternative ranking methods without changing the whole interface.
A practical iteration can be viewed as: requirements clarification → small implementation increment → functional checking → feedback/issue identification → next increment. This keeps

the project focused on demonstrable software rather than documenting features that have not yet been built.

6 Implementation

a. Search Request Flow

The main search interaction begins in the browser. JavaScript sends a POST request to
/api/search with a JSON object containing the entered query. The backend rejects an empty query, identifies concepts, calls the retriever for the top ten positive-scoring resources, obtains related concepts from the relationship index, and serializes the result objects.
The response includes the original query, identified concepts, related concepts, results and result count. Each result includes its identifier, title, author, year, type, subjects, abstract, score, matched concepts, explanation and source note. The frontend stores the returned result list and performs its type/year filtering and newest/oldest sorting locally.

b. Concept Identification

Concept identification is deterministic. The query is lowercased and whitespace-normalized. Canonical vocabulary terms are checked from longest to shortest, and aliases are separately resolved to canonical terms. There is no LLM, embedding model or probabilistic classifier in this module. The same logic is also used to tag catalogue resources from their metadata text.

c. Retrieval and Ranking

The retriever uses scikit-learn's TfidfVectorizer with English stop-word removal. The vector-izer is fitted once on the catalogue's searchable text. For a new query, the query is transformed using that fitted vectorizer and cosine similarity is calculated against the catalogue matrix.
For each resource, the base cosine score is increased by 0.15× the number of concepts shared with the query's identified concepts. Results with a final score greater than zero are retained, sorted by final score in descending order, and truncated to the top ten. This is a classical information-retrieval baseline, not semantic or embedding-based retrieval.

d. Related Concepts

The relationship index records which resources contain each concept. It also counts concept pairs that occur on the same resource. When a query identifies several concepts, the backend aggregates co-occurrence counts for related concepts and returns the highest-scoring related concepts. Selecting one concept calls the concept endpoint, which returns resources tagged with that concept and its most frequent co-occurring concepts.
The relationship structure is therefore a co-occurrence index. It does not represent a learning prerequisite graph, a citation graph, or a graph database.

e. Relevance Explanation

The explanation module uses fixed templates. If a result shares identified concepts with the query, the explanation names those concepts. Otherwise, it uses up to three subject terms when available and otherwise gives a generic overlapping-term explanation. The text is generated from resource metadata and matched concepts; no generative model is involved.

f. Frontend Behaviour

The frontend is contained in frontend/index.html. It includes the visual interface, research-area cards, search box, result toolbar, concept chips, result cards and resource detail modal. Search and concept exploration use the backend API. Filters and sort order are applied to the result list already returned to the browser rather than being sent as query parameters to the backend.
The resource detail UI can display additional fields if they are present in the API response, but the active dataset mainly supplies the core book metadata. Because the current records do not contain TIET biblionumbers, the backend's catalogue URL field is normally absent for search results. The HTML resource view is a locally generated prototype page and explicitly identifies itself as demonstration data.

7 Testing and Evaluation Status

No formal automated unit/integration test suite is included in the supplied prototype. The project README explicitly lists testing as work still to be added. Consequently, this

report does not claim accuracy, precision, recall, F1 score, response-time benchmarks, user-study results, or other quantitative evaluation numbers.
For the current stage, evaluation should be treated as functional verification of the implemented pipeline rather than as a measured search-quality study. The main behaviours to verify in the next iteration are:

1. non-empty and empty query handling;
2. concept extraction and alias matching;
3. TF-IDF ranking and concept boosting;
4. related-concept lookup;
5. type/year filtering and result sorting in the browser;
6. resource-detail retrieval;
7. not-found behaviour for unknown concepts/resources;
8. consistency of the source-data label and prototype-data notice.

The current implementation exposes /api/health, which reports service status, the number of loaded resources, and the data-source label. This is useful for basic service-level checking, but it is not a performance benchmark.

8 Implementation Status and Known Limitations

Current limitation	**Effect / planned direction**
The active dataset is the public *Kaggle Books Dataset*, not a TIET library/repository export.	Replace the isolated loader with a parser for the actual TIET export when it is available.
Retrieval is TF-IDF plus concept boosting.	Keep it as the baseline and compare it experimentally with an embedding-based method before adopting a more complex approach.
The controlled vocabulary is currently fixed in concepts.py.	Rebuild or extend the vocabulary from the real catalogue's subject/keyword fields when that data becomes available.
There is no authentication or per-source access-status labelling.	These are future-scope features rather than current prototype capabilities.
There is no institutional repository or external scholarly-source integration.	Future work may add additional sources once the real data interfaces and access rules are known.
No automated unit/integration test suite is supplied.	Add module-level and API-level tests before treating the prototype as a stable release.
The current source-link fields are placeholders or local	Confirm the real TIET catalogue URL pattern and metadata identifiers before enabling real

prototype routes.	catalogue links.
The relationship model uses plain co-occurrence counts.	A richer relationship model can be evaluated later if the real data and project scope justify it.

9 Risks and Mitigation

Risk	Potential effect	Mitigation
Placeholder data remains in use too long	Results may not reflect actual university resources.	Treat the current dataset as a development stand-in and integrate the real export before any claim about institutional coverage.
Vocabulary misses user wording	Relevant concepts may not be identified.	Maintain aliases and later derive vocabulary from real subject/keyword metadata.
TF-IDF ranks lexical overlap poorly for some queries	A conceptually related item may receive a weak score.	Use TF-IDF as a measurable baseline and compare alternatives experimentally rather than assuming they are better.
Missing metadata/link identifiers	Detail views cannot provide real catalogue access routes.	Validate the real export fields and catalogue URL format before production-facing links are enabled.
Lack of automated tests	Changes can introduce unnoticed regressions.	Add unit tests for each backend module and integration tests for the API flow.

10 Next Steps / Future Scope

1. Receive and integrate the real TIET library/repository data export through the existing data-loading boundary.
2. Rebuild the controlled vocabulary from the real subject and keyword fields where appropriate.
3. Run a baseline evaluation comparing TF-IDF retrieval with an embedding-based alternative on the real data.
4. Add unit and API integration tests for the main modules and endpoints.
5. Conduct a small usability evaluation with students and, if possible, a subject librarian.

6. Refine the ranking and concept-boost weights using evidence from evaluation rather than fixed assumptions.
7. Add authentication/SSO and access-status labels only when the required university integra-tion details are available.
8. Confirm and enable real catalogue/source links once stable identifiers and URL patterns are supplied.

11 Summary

The Academic Research Navigator prototype demonstrates a working discovery pipeline from a natural-language query to ranked resources, relevance explanations and related-concept ex-ploration. Its current implementation is deliberately simple: deterministic concept matching, TF-IDF/cosine ranking with a small concept boost, co-occurrence-based relationships, and template-based explanations.

The most important limitation is also clear: the prototype currently operates on a 6,398-record public book catalogue rather than the intended TIET library/repository data. The system therefore demonstrates the software pipeline, not the coverage or search quality of a live university catalogue. The modular data-loading boundary, together with the separation of concept, retrieval, relationship and explanation logic, provides a practical basis for the next iteration without requiring the current prototype to be presented as a production system.